

Teste de Software

Refatora  o de Testes e Detec  o de Test Smells

Jo o Pedro Campos

Matr cula: 777543

PUCMINAS Lourdes

1. Análise de Smells

A suíte original `userService.smelly.test.js` passava nos testes, mas apresentava problemas de clareza, manutenção e confiabilidade. O primeiro smell identificado foi o Eager Test: o teste deve criar e buscar um usuário corretamente, validava criação e busca no mesmo cenário, misturando responsabilidades e dificultando a identificação da causa da falha.

O segundo smell foi a lógica condicional dentro do teste de desativação. O teste percorria usuários com `for` e decidia os `expects` com `if/else`. Isso torna o teste mais difícil de ler e pode esconder caminhos não exercitados corretamente. O ESLint confirmou esse problema com a regra `jest/no-conditional-expect`.

O terceiro smell foi o Fragile Test no relatório de usuários. O teste dependia de uma linha formatada exatamente, incluindo ID e quebras de linha. Uma pequena mudança de formatação poderia quebrar o teste mesmo sem alterar o comportamento relevante. Também havia tratamento manual de exceção com `try/catch`, que poderia passar silenciosamente se a exceção não fosse lançada, e um teste desabilitado com `test.skip`.

2. Processo de Refatoração

A refatoração foi feita em um novo arquivo, `__tests__/userService.clean.test.js`, mantendo o teste original intacto. Os testes foram reorganizados seguindo Arrange, Act, Assert, com cenários menores e nomes descritivos.

Antes: teste com loop, if e expects condicionais

```
for (const user of todosOsUsuarios) {
  const resultado = userService.deactivateUser(user.id);
  if (!user.isAdmin) {
    expect(resultado).toBe(true);
    expect(usuarioAtualizado.status).toBe('inativo');
  } else {
    expect(resultado).toBe(false);
  }
}
```

Depois: cenários separados e focados

```
test('desativa um usuário comum', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const resultado = userService.deactivateUser(usuarioComum.id);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);
  expect(resultado).toBe(true);
  expect(usuarioAtualizado.status).toBe('inativo');
});

test('não desativa um usuário administrador', () => {
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const resultado = userService.deactivateUser(usuarioAdmin.id);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);
  expect(resultado).toBe(false);
  expect(usuarioAtualizado.status).toBe('ativo');
});
```

Essa mudança elimina a lógica condicional, reduz a complexidade e torna cada teste responsável por uma regra de negócio específica. Também foram criados testes separados para criação, busca, usuário inexistente, relatório vazio, relatório com usuários, menoridade e ausência de dados obrigatórios.

3. Relatório da Ferramenta

O ESLint foi configurado com `eslint-plugin-jest` e executado com `npx eslint ..`. Na primeira análise, a ferramenta automatizou a detecção dos principais problemas do arquivo original:

```
test/userService.smelly.test.js
44:9  error    Avoid calling expect conditionally  jest/no-conditional-expect
46:9  error    Avoid calling expect conditionally  jest/no-conditional-expect
49:9  error    Avoid calling expect conditionally  jest/no-conditional-expect
73:7  error    Avoid calling expect conditionally  jest/no-conditional-expect
77:3  warning  Disabled test                      jest/no-disabled-tests
77:3  warning  Test has no assertions             jest/expect-expect
```

6 problems: 4 errors and 2 warnings

Após a refatoração, o comando `npx eslint __tests__/userService.clean.test.js` foi executado e não apontou erros nem avisos no novo arquivo. A suíte completa também foi executada com `npm test`, e todos os testes passaram, mantendo a funcionalidade preservada.

4. Conclusão

A escrita de testes limpos contribui diretamente para a sustentabilidade do projeto. Testes pequenos, focados e organizados no padrão AAA são mais fáceis de entender, manter e evoluir. Além disso, evitam falsos positivos e reduzem a chance de bugs passarem despercebidos por causa de testes frêgeis ou obscuros.

Ferramentas de análise estática, como ESLint com `eslint-plugin-jest`, ajudam a transformar boas práticas em verificação automática. Elas não substituem a análise manual, mas aceleram a identificação de problemas recorrentes, como `expects` condicionais e testes desabilitados. Combinando análise manual, refatoração e linting, a suíte de testes se torna mais confiável e adequada para apoiar a qualidade do software.

Repositório: adicionar o link do fork após o push para o GitHub.